

Mastery Test 2 - Algorithms and Complexity

Lucas Frykman

1 Problem 1

Lets refer to the n intersections and m as the vertices and edges of a undirected graph $G = (V, E)$. where $V = \{v_i : i \in (1..n)\}$ defining each vertex's state as a portal and and $E = \{(e_i) = (\{u_i, v_i\}, t_i) : i \in 1..m\}$ being a set of weighted edges. My approach will be to reduce this optimization problem in to problem Dijkstra's algorithm can solve. Lets redefine each portal vertex (illustrated as a circle inscribing a square where the square is underworld and the circle is overworld) as sharing an edge and infact lets have every vertex in G be split in to overworld and underworld pairs of vertices.

Algorithm 1 Shortest path with portals

Require: Undirected graph $G = (V, E)$ with edge weights t_e , boolean portal array P , start x , destination y

```
1: Initialize empty vertex set  $\hat{V}$  and edge set  $\hat{E}$ 
2: for each vertex  $v \in V$  do
3:    $\hat{V} \leftarrow \hat{V} \cup v^{(O)}$  ▷ Copy overworld vertex
4:    $\hat{V} \leftarrow \hat{V} \cup v^{(U)}$  ▷ Add its underworld pair
5: end for
6: for each road  $(u, v, t) \in E$  do
7:    $\hat{E} \leftarrow \hat{E} \cup (u^{(O)}, v^{(R)})$  and  $(v^{(O)}, u^{(O)})$  with weight  $t$ 
8:    $\hat{E} \leftarrow \hat{E} \cup (u^{(U)}, v^{(U)})$  and  $(v^{(U)}, u^{(U)})$  with weight  $1/t$ 
9: end for
10: for each vertex  $v \in V$  do
11:   if  $P[v] = 1$  then
12:      $\hat{E} \leftarrow \hat{E} \cup (v^{(O)}, v^{(U)})$  and  $(v^{(U)}, v^{(O)})$  with weight 0
13:   end if
14: end for
15: Run Dijkstra's algorithm from  $x^{(R)}$  in  $\hat{G} = (\hat{V}, \hat{E})$ 
16: return the distance to  $y^{(R)}$ 
```

As seen in algorithm 1, the worst case scenario is if the boolean array P is all 1's i.e every vertex has a portal. In this case we have to insert $2n$ (i.e n extra vertices). The additional portal edges will be $2n$ along with 4 edges that are created for every unordered pair (road) in E . So the total edge count will be at most $|\hat{E}| = 4m + 2n$. The construction of both $\hat{V}$ and $\hat{E}$ runs in $O(n)$. Dijkstra's algorithm will then run in $O((|E| + |V|) \log |V|) = O((|4m + 4n|) \log 4n) = O((m + n) \log n)$. So overall the algorithm will run in $O((m + n) \log n)$.

Correctness Analysis The algorithm correctly finds the optimal minimum travel time by mapping the problem into a standard shortest-path graph representation where every valid physical state is an independent vertex. The edges precisely capture all valid movements and their associated travel times: overworld roads taking t , underworld taking $1/t$, and zero-time portal transitions. Because all travel times are strictly non-negative, Dijkstra's algorithm guarantees finding the global optimal shortest path from the starting intersection to the destination.

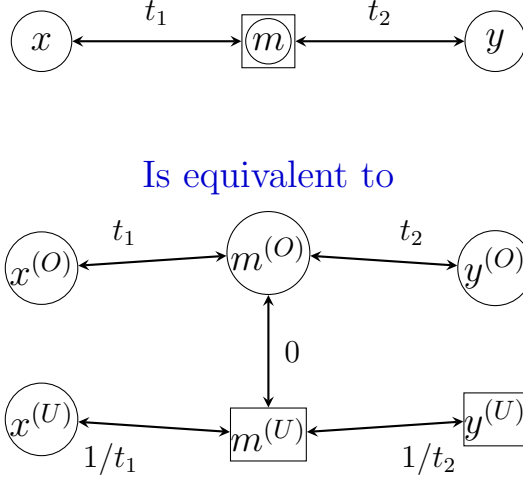


Figure 1: Mapping the information of the first graph to a new graph to handle portal edges

2 Problem 2

We first prioritize the sodas that give the most satisfaction per litre. Define the concentration of soda i as

$$c_i = \frac{s_i}{v_i}.$$

After sorting, assume

$$c_1 \geq c_2 \geq \dots \geq c_n.$$

Let x_i denote the amount of soda i that we drink. The objective function is

$$S(x) = \sum_{i=1}^n c_i x_i - \left(\sum_{i=1}^n x_i \right)^2.$$

The function has negative convexity, since it consists of a linear term minus a penalty in the total consumed volume. Once we restrict attention to a fixed interval where soda i is the marginal soda being consumed, we can find the best amount by setting the derivative to zero.

Suppose we have already consumed a total volume

$$X_{i-1} = \sum_{j=1}^{i-1} x_j.$$

If we now drink x_i litres of soda i , then the derivative of the objective with respect to x_i is

$$\frac{\partial S}{\partial x_i} = \frac{s_i}{v_i} - 2(X_{i-1} + x_i).$$

Setting this equal to zero gives

$$x_i^* = \frac{s_i}{2v_i} - X_{i-1}.$$

Since we cannot drink a negative amount or more than the available volume, we restrict this value to the interval $[0, v_i]$.

Algorithm 2 Greedy algorithm to maximize satisfaction

Require: Satisfaction array $s = [s_1, s_2, \dots, s_n]$ and volume array $v = [v_1, v_2, \dots, v_n]$

1: Sort and reindex the sodas such that

$$\frac{s_1}{v_1} \geq \frac{s_2}{v_2} \geq \dots \geq \frac{s_n}{v_n}$$

2: $X \leftarrow 0$

▷ Total volume consumed so far

3: $S_{current} \leftarrow 0$

▷ Total satisfaction before the penalty

4: $S_{max} \leftarrow 0$

5: **for** $i = 1$ to n **do**

6: $c_i \leftarrow \frac{s_i}{v_i}$

7: $x^* \leftarrow \frac{c_i}{2} - X$

▷ Unconstrained optimal amount of soda i

8: $x^* \leftarrow \min(v_i, x^*)$

▷ Cannot drink more than the available volume

9: $x^* \leftarrow \max(0, x^*)$

▷ Cannot drink a negative amount

10: $X \leftarrow X + x^*$

11: $S_{current} \leftarrow S_{current} + c_i x^*$

12: $S_{max} \leftarrow \max(S_{max}, S_{current} - X^2)$

13: **end for**

14: **return** S_{max}

Correctness Analysis Lets first consider any solution in which some soda i with higher concentration is not fully used while some soda j with lower concentration is used, where

$$\frac{s_i}{v_i} > \frac{s_j}{v_j}.$$

If we move a small amount of volume from soda j to soda i , the total consumed volume remains unchanged. The penalty

$$\left(\sum_{k=1}^n x_k \right)^2$$

does not change. However, the linear satisfaction term increases because soda i gives more satisfaction per litre than soda j . Hence such a solution cannot be optimal.

Thereby there exists an optimal solution that consumes sodas in decreasing order of satisfaction concentration. Specifically the algorithm only needs to consider solutions where we fully consume some prefix of the sorted sodas maybe consume part of the next soda and consume none of the remaining sodas.

Now consider the moment when soda i is the next soda being considered. Let X be the total volume consumed before drinking soda i , and let S_{curr} be the total satisfaction gained before the penalty. If we drink x litres of soda i , the objective value becomes

$$f_i(x) = S_{curr} + c_i x - (X + x)^2,$$

where

$$0 \leq x \leq v_i.$$

This is a concave quadratic function in x since

$$f_i''(x) = -2 < 0.$$

The derivative is

$$f_i'(x) = c_i - 2(X + x).$$

Setting this equal to zero gives

$$x^* = \frac{c_i}{2} - X.$$

The algorithm calculates this before flooring it to v_i or cieling it to 0 if necessary. At every step the algorithm chooses the optimal amount to drink from the current soda assuming all higher concentrated sodas have already been considered. Thus the maximum value stored in $S_{\max}$ is the global maximum. Sorting the sodas by concentration takes $O(n \log n)$ time. The i loop in the algorithm then performs a constant amount of work for each soda, which takes $O(n)$ time. $O(n \log n)$.

3 Problem 3

Algorithm 3 Convert to graph then solve with Dijkstra's algorithm

Require: toxicity array $T = [t_1, t_2, \dots, t_n]$ where t_i is the toxicity of factory i , and filter cost F

- 1: Initialize vertex set $V = \{1, 2, \dots, n + 1\}$ representing filter boundaries (gaps between factories)
- 2: Initialize edge set $E \leftarrow \emptyset$
- 3: **for** $u = 1$ to n **do**
- 4: $t_{max} \leftarrow 0$
- 5: **for** $v = u + 1$ to $n + 1$ **do**
- 6: $t_{max} \leftarrow \max(t_{max}, t_{v-1})$
- 7: $E \leftarrow E \cup \{(u, v, F + (v - u) \cdot t_{max})\}$ $\triangleright$ Add edge for segment containing factories u to $v - 1$
- 8: **end for**
- 9: **end for**
- 10: Find minimum path from vertex 1 to vertex $n + 1$ using Dijkstra's algorithm on graph $G = (V, E)$
- 11: **return** sum of the weights of the edges in the minimum path

Each boundary u needs to be assigned to $n - u + 1$ edges to the factories that succeed it. Which in total will give us $\frac{n(n+1)}{2}$ edges. We only do 2 operations to calculate the weight of each edge (we don't even need to sort the toxicity array) so the time complexity of the graph construction is $O(n^2)$. Dijkstra's algorithm runs in $O((|E| + |V|) \log |V|)$ which is $O(n^2 + n \log n)$ for our graph. Thus the overall time complexity of the algorithm is $O(n^2)$.

Correctness Analysis This algorithm guarantees an optimal solution by framing the filter placement problem as finding the shortest path in a directed acyclic graph. The total cost of each segment relies only on the individual filter installation cost F and the maximum toxicity multiplied by the segment length. By creating a directed edge between every possible pair of start and end boundaries for a segment, weighted by this combined cost, the graph encapsulates all possible valid partitionings of the river. Since all toxicity values and filter costs are non-negative, extracting the shortest path from the first to the last vertex using Dijkstra's algorithm perfectly identifies the partition strategy that yields the minimum total cleanup and installation cost.